PipeRewind

A Time-Travel Debugger for Shell Pipelines

A Beginner-Friendly Project Explanation

Luiz Takahashi & **Ehzoc Chavez**

Washington State University — Systems Programming

April 17, 2026

What You Will Learn

By reading this document you will understand:

- What a shell pipeline is and why debugging one is hard.
- What PipeRewind does and how it solves the problem.
- How the code is organized (even if you have never read C before).
- How to build and use PipeRewind in three simple commands.
- Why each piece of the project matters from a systems-programming perspective.

Reading time: about 10 minutes.

Contents

1 The Problem: Pipelines Are a Black Box **2**
1.1 What is a shell pipeline? . . . 2
1.2 Why is debugging hard? . . . 2

2 The Solution: PipeRewind **3**

3 What Can PipeRewind Do? **5**

4 How to Use It (Step by Step) **6**
4.1 Step 0: Build . . . 6
4.2 Step 1: Record a pipeline . . . 6
4.3 Step 2: Inspect the recording . . . 7
4.4 Step 3: Open the visual replay viewer . . . 7
4.5 Step 4 (optional): Watch a pipeline live . . . 9

5 How the Code Is Organized **9**
5.1 main.c — The front door . . . 9

5.2 pipeline.c — Understanding your command 9
5.3 capture.c — The recording engine 10
5.4 trace.c — The filing cabinet 10
5.5 tui.c — The visual viewer 10
5.6 diff.c — Spot the difference 10
5.7 procstate.c — Health monitor 11

6 How Data Flows Through PipeRewind **11**

7 The Trace File Format **11**

8 Systems Programming Concepts Used **12**

9 Project File Map **13**

10 Quick Reference Card **14**

11 Conclusion **14**

1 The Problem: Pipelines Are a Black Box

1.1 What is a shell pipeline?

In Linux, you can chain programs together with the pipe character (`|`). The output of one program automatically becomes the input of the next:

```
cat access.log | grep ERROR | sort | uniq -c
```

This single line runs *four* programs at the same time:

1. **cat** reads a log file and prints every line.
2. **grep** keeps only lines containing "ERROR".
3. **sort** alphabetizes those lines.
4. **uniq -c** counts how many times each line appears.

Data flows left to right through invisible connections called *pipes*. Each pipe is a one-way tunnel managed by the operating system.

1.2 Why is debugging hard?

Imagine the command above prints nothing. Where is the bug?

- Maybe `cat` could not find the file.
- Maybe `grep` filtered out everything.
- Maybe the data reached `sort` but not `uniq`.

You cannot see what flows between programs because the data travels through the kernel, invisible to the user. This is the core problem PipeRewind solves.

Figure 1: The data moving between commands is invisible — you only see the final output.

2 The Solution: PipeRewind

PipeRewind is a tool written in C that sits between every pair of commands in a pipeline. Think of it like a security camera for your data: it records every piece of text that passes through, stamps it with the exact time, and saves everything to a file. Later, you can "rewind" and replay the entire execution, stepping through it one event at a time — just like scrubbing through a video.

Figure 2: PipeRewind makes everything visible by recording the data between each command.

Key Idea

PipeRewind is **transparent**. It does not change how your pipeline works. The exact same output appears on your screen — but now a complete recording is saved on disk for later inspection.

3 What Can PipeRewind Do?

Figure 3: The three-step workflow: Record, Replay, and Diff.

Here is everything PipeRewind provides at a glance:

Feature	What it does (in plain English)
Record a pipeline	Runs your command normally, but silently saves every byte that flows between stages into a `.prt` trace file.
Replay (TUI)	Opens a full-screen terminal viewer where you can step forward and backward through every event, like scrubbing a YouTube video.
Live mode	Shows the viewer *while* the pipeline is still running, so you can watch data arrive in real time.
Side-by-side diff	Compares what one stage *sent* with what the next stage *received*, highlighting any differences.
Process state tracking	Monitors whether each program is running, sleeping, or finished, so you can spot bottlenecks.
Quoted-argument support	Correctly handles tricky commands like `grep 'hello world'` without breaking on spaces.
Large-trace support	Uses a smart sliding window so huge recordings (100,000+ events) do not use up all your memory.

Table 1: PipeRewind feature summary.

4 How to Use It (Step by Step)

4.1 Step 0: Build

You need a Linux machine with GCC and the ncurses library installed. Then just type:

```
make clean && make
```

This compiles all seven source files and produces a single executable called `piperewind`.

4.2 Step 1: Record a pipeline

```
./piperewind record "echo -e 'banana\napple\ncherry' | sort |
   head -2"
```

You will see the normal output (`apple` and `banana`) on your screen, and a file called `trace.prt` is created silently in the background.

Tip

Use `-v` to see progress messages, or `-o my_trace.prt` to choose a different file name.

4.3 Step 2: Inspect the recording

The simplest way to look at a trace is the `dump` command, which prints a plain-text timeline:

```
./piperewind dump trace.prt
```

Example output (abbreviated):

```
[  0.07 ms] stage 0  PROC_START
[  1.38 ms] stage 0  DATA_OUT  20 bytes  "banana\napple\ncherry\
   n"
[  1.61 ms] stage 1  DATA_IN   20 bytes  "banana\napple\ncherry\
   n"
[  2.44 ms] stage 1  DATA_OUT  20 bytes  "apple\nbanana\ncherry\
   n"
[  2.58 ms] stage 2  DATA_OUT  13 bytes  "apple\nbanana\n"
[  2.80 ms] stage 2  PROC_EXIT exit_code=0
```

Reading this from top to bottom tells you the complete story: Stage 0 produced the unsorted fruit list, Stage 1 (sort) received it and output the sorted version, and Stage 2 (head) kept only the first two lines.

4.4 Step 3: Open the visual replay viewer

For a richer experience, open the interactive TUI:

```
./piperewind replay trace.prt
```

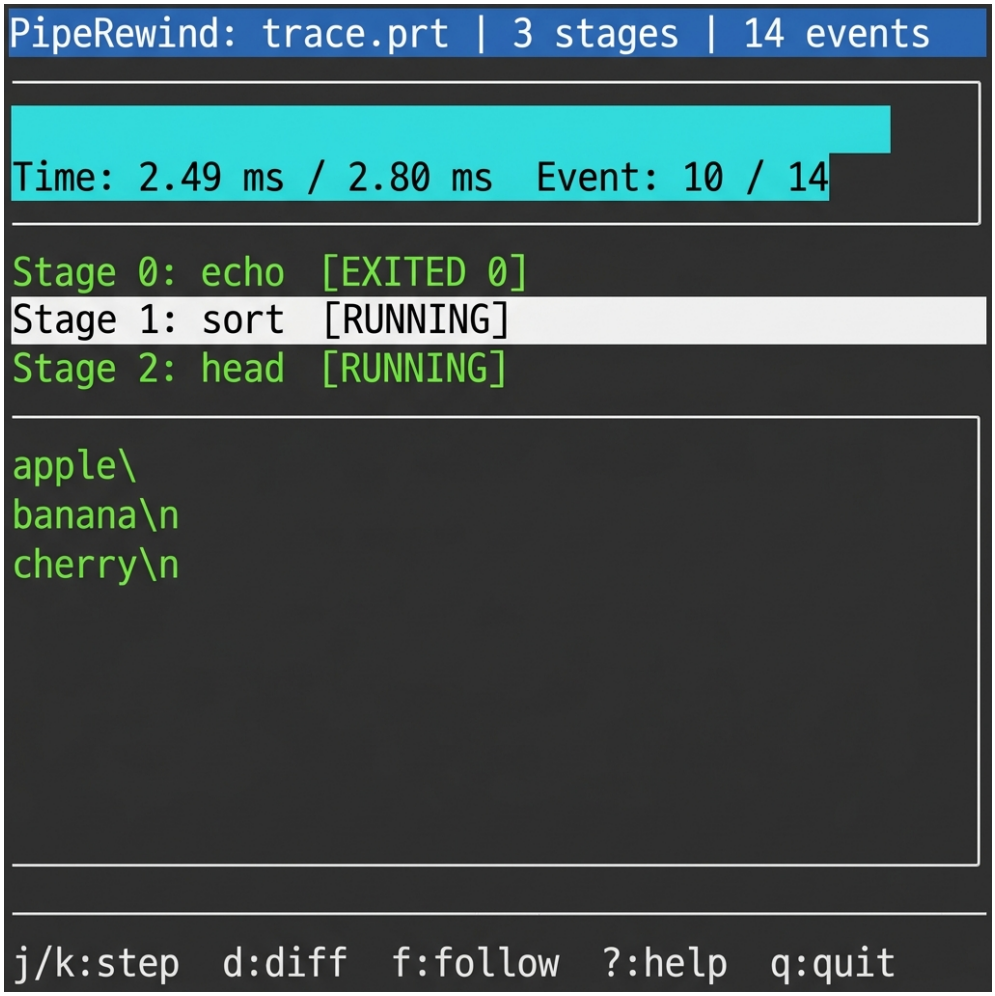

Figure 4: The replay viewer: a full-screen terminal interface where you scrub through time, select stages, and inspect data.

Keyboard controls inside the viewer

Key	Action
`j` / `k`	Step one event forward / backward
`J` / `K`	Jump ten events forward / backward
↑ / ↓	Select a different pipeline stage
`d`	Toggle the side-by-side diff view
`f`	Toggle follow mode (auto-scroll to newest event)
`h`	Switch between text view and hex-dump view
`Home` / `End`	Jump to the first / last event
`?`	Show the help overlay
`q`	Quit

Table 2: TUI keyboard shortcuts.

4.5 Step 4 (optional): Watch a pipeline live

Instead of recording first and replaying later, you can do both at the same time:

```
./piperewind live "tail -f /var/log/syslog | grep error"
```

The viewer opens immediately, and events appear as the pipeline produces them. Press `f` to toggle auto-scrolling.

5 How the Code Is Organized

The project has about 2,500 lines of C spread across seven source files. Each file has one clear job. The diagram below shows how they connect:

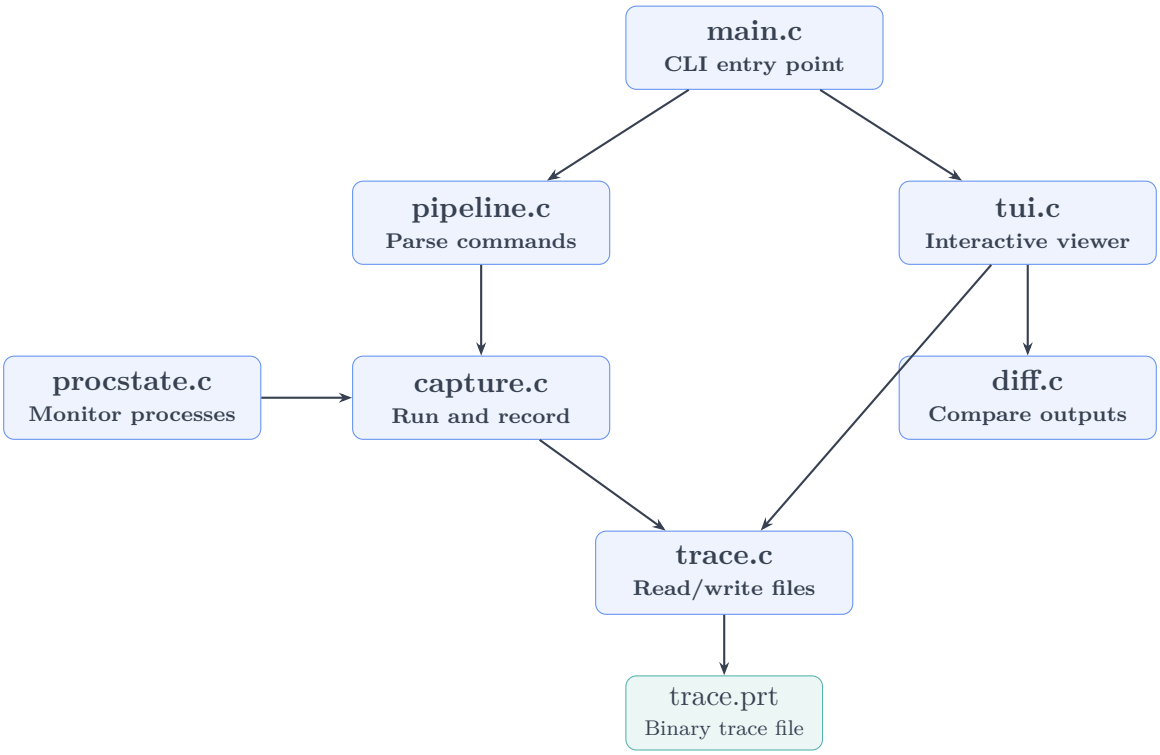

Figure 5: How the source files connect. Arrows mean "uses."

Let's walk through each file:

5.1 main.c — The front door

This is where the program starts. It reads what you typed on the command line (`record`, `replay`, `live`, or `dump`) and calls the right function. Think of it as a receptionist that directs you to the right office.

5.2 pipeline.c — Understanding your command

When you write `"echo hello | sort"`, this file splits that string into separate commands. It is smart enough to handle tricky cases like single quotes (`'hello world'`),

double quotes, and backslash escapes. Under the hood it uses a *state machine* — a pattern where the code keeps track of whether it is currently inside quotes or not, and adjusts its behavior accordingly.

5.3 capture.c — The recording engine

This is the heart of PipeRewind. When you run `record`, this file:

1. Creates child processes (one per command) using `fork` and `exec`.
2. Connects them with pipes, but inserts itself in the middle of each pipe so it can see the data.
3. Uses Linux's `epoll` system to efficiently wait for data from *any* of the pipes without busy-waiting.
4. Every time data arrives, it forwards the bytes to the next command and simultaneously writes a timestamped record to the trace file.

The result is completely transparent: the pipeline behaves exactly the same as if PipeRewind were not there.

5.4 trace.c — The filing cabinet

This file handles reading and writing the binary trace file (`.prt`). It stores a compact header describing the pipeline, followed by a stream of events, followed by an index table for fast lookups. Like a book's table of contents, the index lets the viewer jump to any point in time without reading everything before it.

It also supports *live refresh*: while the capture engine is still writing new events, the reader can detect and load them dynamically.

5.5 tui.c — The visual viewer

The largest file in the project. It uses the `ncurses` library to draw a full-screen terminal interface with:

- A header bar showing the file name and event count.
- A timeline showing how far through the recording you are.
- A list of pipeline stages with color-coded status (green for running, red for exited, yellow for blocked).
- A data panel showing the actual bytes that flowed at the selected moment in time.

It loads events through a *sliding window*: instead of putting the entire trace into memory, it keeps only a chunk of about 20,000 events around your current position, loading more as you scroll.

5.6 diff.c — Spot the difference

When you press `d` in the viewer, this file compares the output of one stage with the input of the next stage, line by line. It uses a well-known algorithm called *Longest Common*

Subsequence (LCS) to find which lines were added, removed, or changed — the same idea used by `git diff`.

5.7 procstate.c — Health monitor

This tiny file reads each process's status from the Linux `/proc` filesystem (the kernel's real-time status board). It reports whether a process is Running, Sleeping, or Blocked, which helps you find bottlenecks.

6 How Data Flows Through PipeRewind

The following diagram shows what happens when you record a two-command pipeline like `echo hello | cat`:

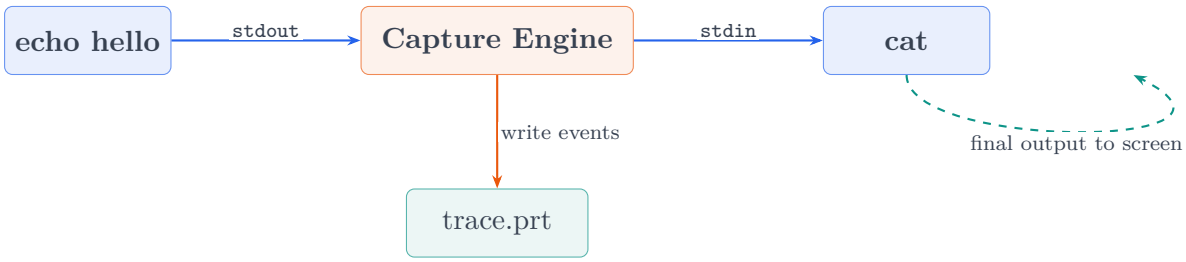

Figure 6: Data flows from `echo` through the capture engine to `cat`. The engine saves a copy to disk.

When data arrives from `echo`:

1. The engine writes a **DATA_OUT** event ("Stage 0 produced these bytes").
2. It writes a **DATA_IN** event ("Stage 1 received these bytes").
3. It forwards the bytes to `cat`'s standard input.

This three-step process repeats for every chunk of data, on every pipe, creating a complete timeline of the pipeline's life.

7 The Trace File Format

The `.prt` file is a custom binary format designed for fast random access. Here is how it is laid out:

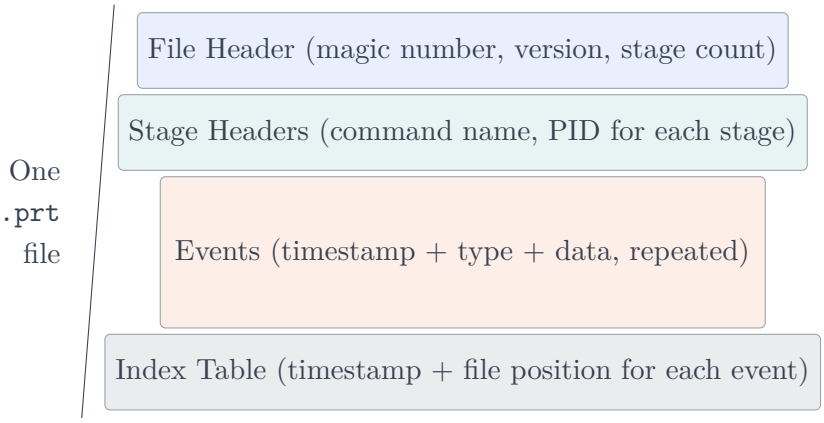

Figure 7: Structure of a `.prt` trace file.

The six event types recorded are:

- **PROC_START** — a process began running.
- **PROC_EXIT** — a process finished (includes exit code).
- **DATA_OUT** — a stage wrote data to its output pipe.
- **DATA_IN** — a stage received data on its input pipe.
- **PIPE_EOF** — a pipe was closed (end of data).
- **PROC_STATE** — a snapshot of whether the process is running, sleeping, or blocked.

8 Systems Programming Concepts Used

PipeRewind is a showcase of key systems programming ideas. Here is how each concept appears in the code:

Concept	Where and how it is used
`fork / exec`	The capture engine creates a child process for each pipeline command with `fork()`, then replaces it with the real program using `execvp()`.
Pipes and `dup2`	Anonymous pipes connect every pair of stages. `dup2()` redirects each child's `stdin`/`stdout` to the correct pipe end.
`epoll` event loop	Instead of checking each pipe one by one (*polling*), the engine asks the kernel to wake it up whenever *any* pipe has data ready, making it very efficient.
`/proc` filesystem	The `/proc/[pid]/status` file is read periodically to check if a process is running, sleeping, or blocked.
Threading (`pthreads`)	In live mode, the capture engine runs on a background thread while the TUI runs on the main thread, sharing the trace file safely.
Binary file I/O	The trace writer uses `fwrite()` for raw binary output, and the reader uses `fseek()` and `fread()` for random access jumps — much faster than a text format for large files.

Table 3: Systems programming concepts in PipeRewind.

9 Project File Map

```
piperewind/
  Makefile                Build instructions (type "make" to
     compile)
  README.md               Quick-start guide
  include/
    capture.h             Capture engine interface
    diff.h                Diff algorithm interface
    pipeline.h            Pipeline parser interface
    procstate.h           Process state monitor interface
    trace.h               Trace file format and reader/writer
       interface
    tui.h                 Terminal viewer interface
  src/
    main.c                CLI entry point (record, replay, live,
       dump)
    pipeline.c            Command-line parser with quoting support
    capture.c             epoll-based capture engine
    trace.c               Binary trace file reader and writer
    tui.c                 ncurses interactive replay viewer
    diff.c                Line-based LCS diff algorithm
    procstate.c           /proc-based process state polling
```

10 Quick Reference Card

Build and test

```
make                # compile
make test           # run automated smoke tests
make clean          # delete build files
```

Record, inspect, and replay

```
./piperewind record "cmd1 | cmd2 | cmd3"    # save trace
./piperewind dump trace.prt                 # text summary
./piperewind replay trace.prt               # visual viewer
./piperewind live "cmd1 | cmd2"             # record + view
```

11 Conclusion

PipeRewind turns an invisible, one-shot pipeline execution into a recorded, rewindable experience. Instead of guessing where data was lost or mangled, you can step through every event, compare outputs side by side, and watch process states in real time.

The project demonstrates core systems programming skills — process creation, pipes, event-driven I/O, threading, and binary file formats — all combined into a practical, usable debugging tool.

Repository: https://github.com/balloonjazz/Pipe-Rewind